PLACE VALUE, NOT POSITION: LENGTH GENERALISATION LIMITS IN NEURAL MODULAR ARITHMETIC

Amey Thakur
Independent Researcher
Toronto, Canada
ameythakur20@gmail.com

11 November 2026

ABSTRACT

A transformer is told where a digit sits, and that choice decides whether it can still add when the numbers get longer. Absolute and rotary encodings describe a digit by its index in the sequence, which moves when an operand is padded; an abacus encoding describes it by its significance, which does not. This paper compares five encodings on one architecture, one dataset, one optimiser and one seed, across three tasks that differ only in the operation: $a + b$, $(a + b) \bmod p$ and $(a \times b) \bmod p$. Every model trains on operands of two to four digits and is scored by exact match on two to eight. Three findings follow. Place value generalises where position does not, but barely: the abacus arms answer 1.4% and 2.1% of problems at unseen widths where both positional arms answer none of $4{,}000$. The gain is small for a reason that has nothing to do with place value. The significance embedding inherits the problem it was meant to solve: only the rows the training widths touch are ever learned. Training that table over a wider range raises out-of-distribution accuracy to 38.3%, with in-distribution accuracy still at 100.0%. And none of it reaches modular multiplication, where all five encodings score between 1.8% and 3.2% inside the training widths and zero beyond, which places the difficulty in the arithmetic rather than the representation. A fourth result concerns the architecture this work submitted to the SAIR Modular Arithmetic Challenge: re-injecting the significance signal at every layer is no better than injecting it once, and on modular addition it is far worse, 24.9% against 76.0%. All code and data are public.

1 Introduction

A calculator given two integers and a prime returns the remainder without thinking. A neural network given the same three numbers has to learn what the remainder is, and the question of whether it can is open in a way the calculator makes easy to forget.

This paper asks a narrower question than whether transformers can do arithmetic. It asks what a transformer is told about *where a digit sits*, and what that choice costs when the numbers get longer than anything the model was trained on.

The reason the question has teeth is a structural mismatch, and Figure 1 is the whole of it in one picture. A transformer ordinarily learns an embedding for each position in its input, so the token at index seven is described by whatever the model learned about index seven. Under that description, padding an operand by one digit moves every digit in it to a new index, and a model trained on four-digit operands has no description at all for index nine. The alternative is to describe a digit by its *significance*, the power of the base it carries, which is a property of the digit rather than of the string it sits in. The units digit has significance zero whether the number has two digits or two hundred.

That distinction is not a trick for arithmetic. It is the difference between a description that survives what the task does to its inputs and one that does not, and this paper measures what it is worth.

1.1 Contributions

1. **A controlled comparison of five encodings** on the same architecture, data, optimiser and seed, differing in nothing but how a token's place is described (Section 4). Two describe position in the sequence and three describe place value, differing in where the signal enters and whether its embedding is trained beyond the widths the data provides.
2. **A three-rung difficulty ladder** that separates the cost of the encoding from the cost of the arithmetic (Section 4.4). Plain addition, modular addition and modular multiplication share every experimental setting and differ only in the operation, so a fall in accuracy between rungs is attributable to the arithmetic alone.
3. **A measurement of what place value is worth, and where.** Describing a digit by its significance rather than its index buys accuracy beyond the training widths where a positional description buys none at all, 1.4% and 2.1% against 0 of $4{,}000$ (Section 6). It buys considerably more inside the training widths once the task is hard enough to need it: 76.0% against 44.8% on modular addition. The gain is real, and it is one digit wide.
4. **Two negative results, one of them about this work's own submission.** Re-injecting the significance signal at every layer, the architecture submitted to the competition, is no better than injecting it once and is far worse on the harder rung, 24.9% against 76.0% (Section 7.2). And modular multiplication, the competition's own operation, defeats every encoding equally: all arms sit between 1.8% and 3.2% in distribution and at 0 of $4{,}000$ beyond it, which places the difficulty in the arithmetic rather than in the representation.

1.2 Hypotheses

Four claims organise the experiments. Each one admits a measurement that could contradict it, and Section 7.6 returns to each with the evidence that bears on it.

H1 (place value generalises). A model told the significance of a digit answers correctly on operand widths it never saw in training, where a model told the index of a digit does not.

H2 (re-injection helps). Adding the significance signal at every layer beats adding it once at the input, because a residual stream that carries many other things dilutes a signal injected only at the start.

H3 (the operation binds, not the encoding). Moving from addition to modular multiplication costs more accuracy than any choice of encoding recovers, so the encoding is not what stands between these models and the competition's task.

H4 (in-distribution accuracy does not buy extrapolation). A model can be accurate at the widths it trained on and still fail immediately beyond them, so reporting in-distribution accuracy alone hides the failure this paper is about.

H5 (the table must be trained wider than the data). A significance embedding whose rows are exercised only over the training widths extrapolates no better than a positional one, because the rows it needs beyond them are untrained. Shifting significance during training should recover the difference.

The measurements support H1, H3 and H4. They refute H2, and the refutation is the sharper finding: the architecture this work submitted is measurably worse than the simpler one it was meant to improve on. H5 was formed after the first four arms had run, and Section 7.6 settles it last.

2 Background

2.1 The task, and why exact match is the right score

Fix a prime p and two integers a and b. The problem is to produce

$$(a \times b) \bmod p$$

exactly. The SAIR Modular Arithmetic Challenge sets it at a scale where the operands run to roughly 1,233 decimal digits and the modulus to roughly 617, with a and b permitted to exceed p, and everything presented as decimal strings (Alfarano et al., 2026).

Scoring is exact match, and that is not a stylistic choice. A remainder that is off by one is not nearly right; it is a different residue, and every use of modular arithmetic downstream, in cryptography above all, treats it as simply wrong. A model that is accurate to within one part in 10^{600} has produced nothing of value. Most sequence prediction rewards a near miss; this problem does not, and that shapes everything below.

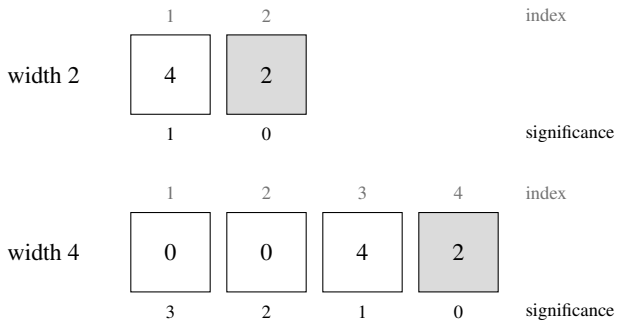


Figure 1: The same number at two widths, as a padded input reaches a transformer. The shaded cell is the units digit in each row. Its index in the sequence moves from 2 to 4, so whatever a model learned about index 2 no longer applies to it; its significance is 0 in both rows, so a model told the significance recognises it from every example it has ever seen. Widening an operand moves every index and leaves every significance where it was, which is the whole of the argument this paper tests.

2.2 Position and place value

Write an integer n in base B as

$$n \;=\; \sum_{i=0}^{w-1} d_i \, B^{\,i}, \qquad 0 \le d_i < B,$$

so that d_i is the digit of *significance* i. The pair (d_i, i) determines the digit's contribution to the value of n and does so independently of w, the width of the number, and independently of where the number appears in any string.

A transformer does not see this. It sees a sequence of tokens and, in the usual construction of Vaswani et al. (2017), adds to each an embedding of its index in that sequence. Let $\mathrm{idx}(d_i)$ be that index. For a number written most significant digit first at offset c in the sequence,

$$\mathrm{idx}(d_i) \;=\; c + (w - 1 - i),$$

which depends on w. Widen the operand by one digit and every index shifts. Train on $w \le 4$ and the model has seen indices up to $c + 3$ and has learned nothing about $c + 4$.

The significance i has no such dependence. The abacus embeddings of McLeish et al. (2024) rest on that observation. The experiments below ask what it is worth once a modulus is involved.

2.3 What transformers are known to do with arithmetic

Three findings frame the question. Small transformers trained on modular addition with a *fixed* small modulus do learn it. They do so late: long after the training loss has settled, they transition from memorising to computing, a phenomenon Power et al. (2022) named grokking. Nanda et al. (2023) later traced it to a specific Fourier construction in the weights. Length generalisation in arithmetic is substantially a question of positional encoding, and can be improved markedly by changing that encoding rather than the architecture (McLeish et al., 2024; Ruoss et al., 2023). And the class of algorithms a transformer learns to extrapolate is narrow, and predictable from whether the task can be written as a short RASP program (Zhou et al., 2023).

Against these sits a fourth finding, which is the one this paper takes most seriously. Lauter et al. (2024) applied both circular regression and sequence-to-sequence transformers to modular multiplication and reported that both largely failed, and read that failure as evidence for the hardness on which lattice cryptography rests. Two of those authors organise the competition this paper's task comes from.

2.4 The competition

The SAIR Modular Arithmetic Challenge ran from 8 June to 12 August 2026 under the SAIR Foundation, organised by Alberto Alfarano, François Charton, Yongzheng Jia, Kristin Lauter, Cathy Li, Terence Tao and Emily Wenger (Alfarano et al., 2026). A submission is a public model repository identified by a commit hash, which the organisers evaluate against a secret seed.

The rules are written so that the obvious shortcuts are structurally impossible rather than merely forbidden. Three preprocessing hooks see a, b and p separately, so no point in submitted code ever holds all three at once; arbitrary-precision arithmetic on the original arguments is disqualifying, as are lookup tables keyed on the inputs, dynamic evaluation, network access and subprocesses. The effect is that a submission must compute the answer with learned parameters, and the competition is a test of what those parameters can hold.

3 Related work

The findings of Section 2.3 are taken up here one at a time.

Grokking and algorithmic generalisation. Power et al. (2022) showed that small transformers on small algorithmic datasets generalise long after they have fit the training set, and that weight decay governs the transition. Nanda et al. (2023) identified the mechanism for modular addition, a discrete Fourier transform implemented in the attention and MLP weights, and gave progress measures that track its formation. Both study a fixed modulus, which makes the task a finite group operation the model can internalise. The experiments here give the modulus as input, which removes that possibility and is the harder setting.

Positional encoding and length generalisation. Relative and rotary schemes were introduced to improve extrapolation (Su et al., 2021; Press et al., 2022), and Ruoss et al. (2023) showed that randomising positions during training improves it further by preventing a model from binding behaviour to particular indices. McLeish et al. (2024) took the step this paper builds on, encoding a digit's significance rather than its index, and reported large gains on addition. Zhou et al. (2023) give the complementary negative account: the tasks that extrapolate are those expressible as short RASP programs, which predicts that a task requiring a long sequential carry chain will not.

Transformers on number-theoretic functions. Charton (2024) trained transformers to compute the greatest common divisor and was able to characterise the learned predictor completely, finding that the model learns a list of divisors of the base and predicts the largest that divides both inputs. That result is a caution as much as a success: a transformer that appears to do number theory may be doing something much simpler that agrees with number theory on the training distribution. Africa et al. (2025) study modular exponentiation and find interpretable circuits in the final attention layer.

Machine learning against modular arithmetic in cryptography. The motivation for much of this work is cryptanalytic. Wenger et al. (2022) trained transformers to perform modular arithmetic as a component of an attack on learning-with-errors schemes, the first use of transformers against lattice cryptography. Lauter et al. (2024) addressed modular multiplication directly and reported limited success, which they interpret as evidence for the hardness that such schemes rely on. The present paper measures that boundary from the other side, holding the encoding, the architecture and the data fixed and varying only the operation, so that the difficulty can be attributed.

Intermediate computation. Asking a model to emit its working converts a fixed-depth network into something closer to a recurrent one, and buys accuracy on multi-step arithmetic (Nye et al., 2021; Wei et al., 2022). The repository behind this paper implements scratchpads in the manner of Horner's method (Thakur, 2026c). They are not varied in the experiments below, because doing so would confound the encoding comparison; Section 8 returns to what that leaves unmeasured.

Positioning within the author's prior work. Several strands of the author's earlier work bear on the design adopted here. The Galois group classifier of Thakur (Thakur, 2026a) meets the same shape of problem from the opposite direction. There the evidence is cheap, and the question is how far a statistic can be pushed before the candidates become indistinguishable. The answer took the form of a bound computed before any data was seen. The present paper looks for the analogous boundary, and finds it in the operation rather than in the representation. The gesture detector of Thakur (Thakur, 2026b) identifies a class from a deterministic structural signature rather than a learned one. The significance of a digit is exactly such a signature: computed, not learned. It is what the abacus arms receive and the positional arms do not. The treatment of graded confidence in Thakur, Dhiman and Phansikar (Thakur et al., 2021a) is relevant here by contrast: exact match admits no graded answer, which is what Section 2.1 argues sets this task apart from most sequence prediction. The classification background, the decoder, the embedding table and the optimiser every arm holds fixed, follows Thakur and Konde (Thakur and Konde, 2021).

Four further strands bear on it more obliquely, and are named because each supplies a piece of the design rather than because they are the author's. Arithmetic modulo p is the arithmetic of a clock face, and the setting where that arithmetic is ordinarily met in systems work is keeping separated clocks agreed (Thakur and Satish, 2022); the wrap-around that

makes a clock's arithmetic modular is the same discontinuity that Section 4.4 isolates as the step which defeats every encoding tried here. A quadtree addresses a point by recursive subdivision, so that each digit of the address names a scale rather than a position in a string (Thakur et al., 2022); the significance embedding of Section 4.2 is that idea applied to the digits of a number instead of the coordinates of a plane, which is why it survives padding when an index into the sequence does not. Every arm here is trained by cross-entropy against the correct digit, a maximum-likelihood objective, and the contrast with the adversarial objective surveyed in Thakur and Satish (2021a) is what makes the exact-match score of Section 2.1 available: a likelihood-trained decoder yields a calibrated distribution at each position, which a generator trained against a discriminator need not. Finally, two image-translation systems (Thakur et al., 2021b; Thakur and Satish, 2021b) make this paper's central claim in a different medium. What a model carries to inputs it was not trained on is fixed by the representation it is handed, not by its capacity: a decomposition into surface, structure and texture in the first, and open-domain adaptation from a sketch in the second.

4 Method

4.1 What is held fixed

The five encodings of Section 4.2 are compared under the protocol of Section 5. Every arm uses one decoder-only transformer: three layers, four heads, model width 128, feed-forward width 256, no dropout, trained with AdamW at a peak learning rate of 3×10^{-4} under a one-cycle schedule with weight decay 0.1. Tokenisation is character-level over the ten digits and a handful of separators, with no sub-word merges, which would otherwise split a number into fragments that do not respect place value.

Each arm receives exactly one extra embedding table beyond the token embeddings, so parameter counts match to within the difference between a position table and a significance table, and no arm has more capacity than another.

4.2 The five encodings

Figure 2 shows where each description of place enters the network, which is the whole of the difference between the arms.

Absolute. A learned embedding of the token's index in the sequence, added once at the input. This is the construction of Vaswani et al. (2017) with a learned rather than sinusoidal table.

RoPE. Rotary position encoding (Su et al., 2021), applied to queries and keys at every layer. Still a description of sequence position, but a relative one.

Abacus, input. A learned embedding of the digit's significance, its place value inside its own number, added once at the input. Non-digit tokens receive a slot of their own.

Abacus, injected. The same significance embedding, re-added to the residual stream before every layer. This is the architecture the repository submitted to the competition, and isolating it from the input-only variant is what makes H2 answerable.

Abacus, offset. The input variant again, but during training every digit's significance is shifted by one random amount drawn afresh for each batch, so that rows of the embedding table far beyond the operand widths receive gradients. Evaluation always uses the true significance. This arm was added after the first four had run, for the reason Section 4.3 gives, and it differs from *abacus, input* in the training shift and in nothing else.

The first four arms were fixed before any measurement. The fifth is a response to what the first four showed, and is reported as such rather than presented as part of the original design.

4.3 An embedding that is never trained

There is a way for the significance encoding to fail that has nothing to do with whether place value is the right description. The embedding table is learned, and only the rows the training data touches receive gradients. Training on widths $w \in \{2, 3, 4\}$ exercises significance 0 through 3 and no others. A model asked at width 8 for the digit of significance 7 reads a row that has never been updated, and is therefore reading noise.

Described this way, the abacus arms inherit the very problem they were introduced to solve: a description the model has never seen is no more useful than an index the model has never seen. The remedy, due to McLeish et al. (2024), is to shift the significance indices by a random amount during training, so that the table is exercised over a range far wider than the operand widths alone would reach. The *abacus, offset* arm applies exactly that shift and changes nothing else, so the difference between it and *abacus, input* measures the cost of leaving those rows untrained.

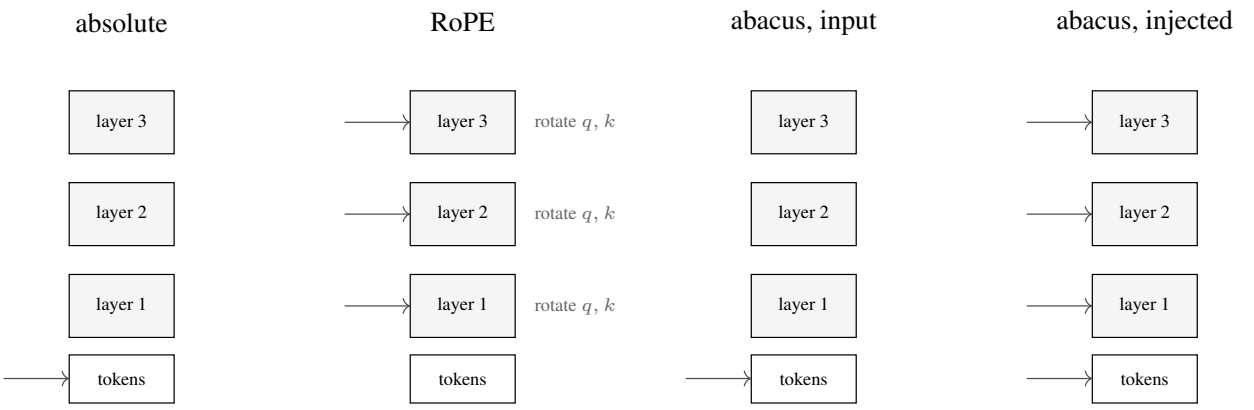


Figure 2: The four encodings, which differ only in what a token is told about its place and where it is told. The first two describe position in the sequence, absolutely or, for rotary encoding, by rotating queries and keys at every layer. The last two describe the digit's significance, once at the input or re-added before every layer. Everything else, the layers, the widths, the optimiser and the data, is identical across the four. A fifth arm, *abacus, offset*, is not drawn because it shares this architecture exactly with *abacus, input* and differs only in how its embedding is trained, which Section 4.3 explains.

Rung	Operation	Example at width 3	What the reduction costs
1	$a + b$	$471 + 638 = 1109$	nothing; no modulus
2	$(a + b) \bmod p$	$(471 + 638) \bmod 683 = 426$	a comparison, one subtraction
3	$(a \times b) \bmod p$	$(471 \times 638) \bmod 683 = 661$	a division

Table 1: The difficulty ladder. Operands are drawn below the modulus on rungs 2 and 3, so the answer is a residue of the same width. The examples are genuine: 683 is prime, and each line is the arithmetic a model on that rung must reproduce exactly. Only the operation changes between rungs, so a fall in accuracy from one to the next is attributable to the arithmetic and to nothing else.

4.4 The difficulty ladder

Three tasks, differing only in the operation, set out in Table 1:

1. $a + b$, with operands of width w and an answer of width $w + 1$. No modulus. This is the canonical length-generalisation benchmark, which anchors the experiment against published results rather than leaving it to stand alone.
2. $(a + b) \bmod p$, with $a, b < p$ and p prime of width w. The reduction is a comparison and at most one subtraction.
3. $(a \times b) \bmod p$, the competition's own operation, under the same conditions. The reduction is now a division.

Operands are written zero-padded to the modulus width, so a digit of a given significance sits at a fixed sequence index for a fixed width. That is the most generous possible setting for the positional arms. Within a width they could in principle memorise the layout; only a change of width disturbs them.

The answer is emitted least significant digit first, so that generation order matches the direction carries travel. Writing it the other way round asks a causal decoder to know a carry before it has computed it. That single choice separates a task that trains from one that sits at chance (McLeish et al., 2024). Significance is unaffected by this choice, being a property of the digit rather than of where it is written.

5 Experimental setup

Data. Models train on operand widths $\{2, 3, 4\}$ and are scored on $\{2, \ldots, 8\}$, so widths 5 and beyond are out of distribution. Primes are generated per width by trial division and shared across arms. Each arm sees 20,000 problems per training width, drawn in the same order from the same seed, so no arm sees data another did not.

Held-out problems. Evaluation problems are generated after the training set from the same stream, and any that appear verbatim in training are discarded, so a model cannot score by recitation. One thousand problems are scored at each width, except where deduplication leaves fewer, which happens only at the narrowest width and for the reason given below.

How much of each space training sees. The widths are not equally informative, and the reason is arithmetic rather than experimental. A width-w addition problem is an ordered pair of w-digit operands, so its space has $(9 \cdot 10^{w-1})^2$ members. A width-w modular problem is a triple (a, b, p) with p one of the w-digit primes the generator drew and $a, b < p$, so its space has $\sum_p p^2$ members over those primes. Both counts, and the fraction of each that 20,000 draws per width actually reaches, are in Table 2; `tools/problem_space.py` recomputes them by replaying the generator, and reproduces the held-out counts of the published run exactly.

Width 2 is the width at which recall is available. Training visits 91.4% of the addition space there and 22.0% of the modular one, so an accuracy at that width reports how much of a small space a model can hold as much as whether it has learned to compute, and after removing problems that appear verbatim in training only 132 held-out addition problems and 851 held-out modular problems remain. By width 4 coverage is two parts in ten thousand for addition and six in a million for modular multiplication. A perfect score there cannot be recall, which is what makes the behaviour immediately beyond it worth measuring.

	$a + b$		$(a \times b) \bmod p$	
Width	reachable	reached	reachable	reached
2	8,100	91.4%	65,709	22.0%
3	810,000	2.4%	28,174,903	0.07%
4	81,000,000	0.02%	3,492,251,466	0.0006%

Table 2: Distinct problems a training width can present, and the fraction of them that the 20,000 draws at that width actually reach. Modular addition draws from the same space as modular multiplication. The right-hand column is why width 4 is the width the argument rests on: nothing there can be answered from memory.

Aggregates. Accuracy at a single width is exact match over that width's held-out problems. An accuracy quoted for a range of widths is *pooled*: correct answers over the range divided by problems in it, which is the fraction of held-out work the model got right. It is not the unweighted mean of the per-width accuracies. The two differ only where the held-out sets differ in size, which happens only at width 2, and the gap is immaterial for every arm but one: rotary encoding on plain addition is 58.2% pooled and 70.3% unweighted, because it is perfect at widths 2 and 3, scores 10.9% at width 4, and the 132 held-out problems at width 2 carry about an eighth of the weight of the 1,000 at width 4. Every interval in this paper is a Wilson interval at 95% on the pooled count.

Scoring. Accuracy is exact match on the whole answer. Digits are decoded greedily and fed back, so an early mistake is not repaired later, which is the behaviour the competition scores.

Budget. Each of the fifteen configurations, three tasks by five encodings, trains for 6,000 steps at batch size 256. All fifteen run on CPU.

6 Results

Fifteen configurations were trained, three task rungs by five encodings, each for 6,000 steps under the protocol of Section 5. Figure 3 shows exact-match accuracy against operand width for every one of them, with the widths never seen in training shaded. Table 3 splits the same measurements into the training range and beyond it, and Table 4 collapses them by operation to show what each rung of the ladder costs.

6.1 Addition is learned, and does not survive one extra digit

On plain addition four of the five encodings reach 100.0% exact match across the training widths, and every one of them scores zero on the first width they have not seen. Absolute positions and all three abacus variants solve widths 2 to 4; rotary positions do not, and Section 6 returns to why below. At width 5 the positional arms answer 0 of 4,000 held-out problems correctly, an interval of $[0.000, 0.001]$.

The width-4 figure is the one that matters. There are 81,000,000 distinct four-digit addition problems and training sees almost none of them twice, so a perfect score there cannot be recall: the models have learned to add. One digit further, having learned to add, they cannot.

The abacus arms do not escape this, but they do not fall to zero either. Over the four unseen widths they answer 54 and 82 of 4,000 correctly, 1.4% $[0.010, 0.018]$ for input injection and 2.1% $[0.017, 0.025]$ for per-layer injection. Both intervals exclude the positional result. The effect is real, and it is small: almost all of it sits at width 5, and by width 7 every arm is at zero.

Rotary encoding is the exception in distribution, reaching only 58.2% $[0.561, 0.603]$ because it never finished learning width 4, where it scores 10.9%. Its zero beyond the training range therefore carries less weight than the others: a model that has not learned the task cannot be said to have failed to extrapolate it. Section 8 returns to this.

6.2 Training the table wider than the data

The fifth arm changes the first rung completely. Shifting significance by a random amount during training, and evaluating on the true significance, leaves in-distribution accuracy at 100.0% and raises out-of-distribution accuracy from 2.1% to 38.3% $[0.368, 0.398]$, which is 1,532 of 4,000 against 82. The intervals are nowhere near one another.

The gain is concentrated where it can be: 87.6% at width 5, 46.6% at width 6, 18.6% at width 7 and 0.4% at width 8. Accuracy decays with distance from the training widths rather than collapsing at the first step beyond them, which is the behaviour every other arm fails to show.

On the modular rungs the same change does not pay. Modular addition falls to 32.9% $[0.312, 0.347]$ in distribution, well below the 76.0% of a single input injection, and gains almost nothing beyond, 14 of 4,000 against 13. Modular multiplication is unmoved: 2.4% in distribution and 0 of 4,000 outside it, indistinguishable from the arms it was meant to improve on.

6.3 The modular reduction separates the encodings

Adding a modulus changes the ranking. Where plain addition was solved by three arms alike, $(a + b) \bmod p$ is solved best by significance at the input, at 76.0% $[0.744, 0.776]$ across the training widths, against 63.6% for rotary, 44.8% for absolute positions and 24.9% $[0.234, 0.266]$ for per-layer injection. At width 4 the spread is starker still: 56.4% for abacus input against 2.3% for absolute positions.

Place value therefore buys something in distribution once the task is hard enough to require it, which the first rung was not. Out of distribution the picture is the same as before but fainter: abacus input answers 13 of 4,000, and every other arm answers none.

6.4 Multiplication defeats every encoding

The third rung removes the differences entirely. Across the training widths the five arms score 3.1%, 3.2%, 2.9%, 1.8% and 2.4%, intervals that overlap one another, and beyond the training widths all five answer 0 of 4,000.

The in-distribution figures are themselves generous. Almost all of that small success sits at width 2, where the five arms score between 5.2% and 9.8% and where, by Table 2, training has already visited 22.0% of the reachable problems. By width 4, where it has visited six in a million and recall is therefore unavailable, every arm is at 0.0%. Nothing here has learned to multiply and reduce, and what little it has sits exactly where memory is cheapest.

7 Analysis

Section 6 reports what the fifteen models do, and Figure 4 shows all fifteen at every width at once. Read as a picture rather than as a list of numbers, it makes three claims that the rest of this section takes in turn: the failure at an unseen width is a cliff and not a slope, exactly one arm has anything left after it, and one operation is empty everywhere including inside the training range. This section asks what the pattern means for the three questions the hypotheses

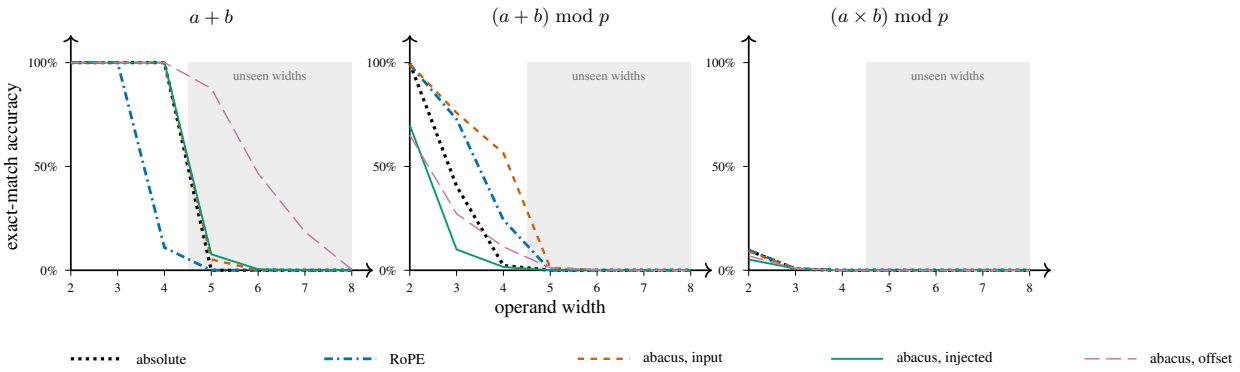


Figure 3: Exact-match accuracy against operand width, for each task and each encoding. Models trained on widths 2 to 4; the shaded band is the widths they never saw. The horizontal axis is operand width and the vertical axis accuracy. Each arm keeps one colour and one dash pattern throughout, so the figure reads in greyscale as well as in colour; where arms coincide, as four of them do at 100% across the training widths of the left panel, the later ones are drawn thinner and show through as a fringe. Panels share every setting except the operation, and curves within a panel share every setting except how a token's place is described.

Encoding	Widths 2–4	Widths 5–8	Best unseen width
$a + b$			
absolute	100.0%	0.0%	0.0%
RoPE	58.2%	0.0%	0.0%
abacus, input	100.0%	1.4%	5.2%
abacus, injected	100.0%	2.1%	7.7%
abacus, offset	100.0%	38.3%	87.6%
$(a + b) \bmod p$			
absolute	44.8%	0.0%	0.0%
RoPE	63.6%	0.0%	0.0%
abacus, input	76.0%	0.3%	1.2%
abacus, injected	24.9%	0.0%	0.0%
abacus, offset	32.9%	0.4%	1.2%
$(a \times b) \bmod p$			
absolute	3.1%	0.0%	0.0%
RoPE	3.2%	0.0%	0.0%
abacus, input	2.9%	0.0%	0.0%
abacus, injected	1.8%	0.0%	0.0%
abacus, offset	2.4%	0.0%	0.0%

Table 3: Exact-match accuracy inside and outside the training widths, pooled over the test problems of each range in the sense of Section 5. The middle column covers the four widths the model never saw, and the right-hand column the best single unseen width, which separates a model that degrades gradually from one that fails everywhere beyond its training range.

of Section 1.2 pose: whether place value buys extrapolation, whether re-injecting it helps, and whether any encoding reaches the operation the competition actually sets.

7.1 What the encoding buys, and where

Read down the rungs, the encoding matters in a narrow band. On plain addition it is nearly irrelevant in distribution, because four of five arms reach 100.0% and the fifth is held back by convergence rather than by its description of place.

	Widths 2–4		Widths 5–8	
Operation	mean	best	mean	best
$a + b$	91.6%	100.0%	8.3%	38.3%
$(a + b) \bmod p$	48.5%	76.0%	0.1%	0.4%
$(a \times b) \bmod p$	2.7%	3.2%	0.0%	0.0%

Table 4: What each rung of the ladder costs, across the five encodings. Each entry is taken over the five arms of its row: 'mean' is their average and 'best' the strongest of them, both read off the pooled accuracies of Table 3. The operation is the only difference between rows; every other setting is identical. A row that is low in both columns is low for reasons no encoding fixes.

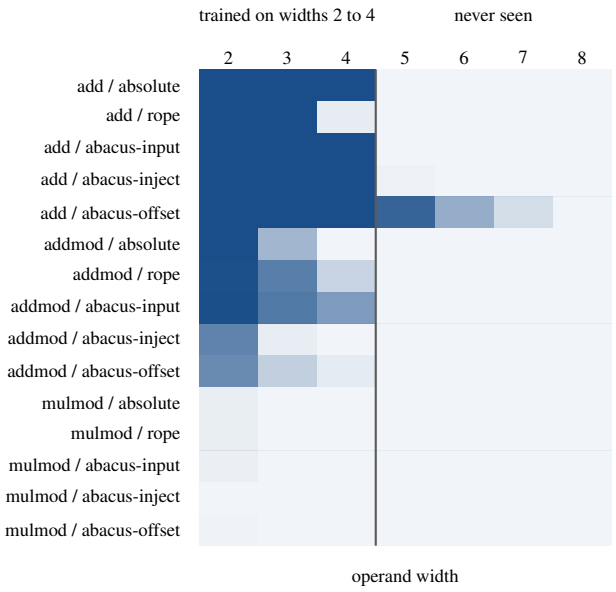


Figure 4: Exact-match accuracy for every arm at every tested width, with opacity proportional to accuracy. The vertical rule marks the edge of the training range. Three things are visible at once that the prose can only state one at a time: almost every arm falls off a cliff at the first unseen width rather than degrading, exactly one arm crosses it with anything left, and the bottom third of the figure is empty at every width including the trained ones, which places the difficulty of modular multiplication in the operation and not in the encoding. Cells are generated from `data/length_generalization.json` and verified against it in continuous integration.

On modular addition it matters a great deal: the spread across arms at width 4 runs from 2.3% to 56.4%. On modular multiplication it does not matter at all, because nothing works.

The band is therefore the middle rung, where the task is hard enough that a stable description of a digit pays and easy enough that the model can still learn it. Outside that band the encoding is either unnecessary or insufficient, and neither case is visible from the first rung alone.

7.2 Re-injection did not help, and on the harder rung it hurt

Adding the significance signal before every layer is the architecture this work submitted to the competition, and it is worse than adding it once.

Out of distribution on plain addition the two variants are indistinguishable, 1.4% $[0.010, 0.018]$ against 2.1% $[0.017, 0.025]$, intervals that overlap. On modular addition the difference is large and in the wrong direction: 76.0% $[0.744, 0.776]$ for a single injection at the input against 24.9% $[0.234, 0.266]$ for injection at every layer, intervals

nowhere near each other. Out of distribution on the same rung, input injection answers 13 of 4,000 and per-layer injection none.

The reasoning behind H2 was that a residual stream carrying many things dilutes a signal added once at the start. The measurement says the opposite matters more. Re-adding a fixed vector at every layer competes with whatever the residual stream has computed so far, and on a task that needs several steps of computation, that interference costs more than the dilution it prevents. The hypothesis was stated before the measurement and the measurement refutes it.

7.3 Fitting the training widths predicts nothing about the next one

Three arms reach exactly 100.0% on plain addition across the training widths and then score 0.0%, 1.4% and 2.1% beyond them. An in-distribution number, however clean, orders the arms differently from the out-of-distribution number it is supposed to predict.

The clearest case is within the abacus pair. Adding plain numbers, the input-injection arm reached a lower training loss than the per-layer arm and a slightly lower out-of-distribution score; on modular addition it reached both a higher in-distribution score and a higher out-of-distribution score. Neither quantity tracks the other reliably, which is the whole of H4 and the reason every table here reports the two separately.

7.4 The embedding, not the idea, was the bottleneck

The gap Section 6.2 reports, between 2.1% and 38.3% on plain addition, is the paper's largest effect, and the two arms either side of it differ in nothing but which rows of one embedding table received gradients. The architecture is the same, the data is the same, the seed is the same. What changed is that significance 4 through 7 were trained.

That settles what the first four arms could not distinguish. The modest extrapolation of the abacus arms was not evidence that place value is a weak idea; it was evidence that an unlearned description is useless whatever it describes. A significance embedding trained only over the widths in the data reproduces, one level down, exactly the failure that motivated abandoning absolute positions.

The qualification matters as much as the result. The same change costs 43 points of in-distribution accuracy on modular addition and buys nothing beyond it. Shifting significance makes the training distribution harder, since the model must handle every offset rather than one, and under a fixed budget that cost is only worth paying when the task is otherwise easy enough to absorb it. On the first rung it is; on the second it is not; on the third nothing is.

7.5 Why multiplication is different

Addition of two w-digit numbers is a single pass with a carry, and a transformer can represent it. Modular addition adds a comparison and at most one subtraction. Modular multiplication is neither: it requires a product of $2w$ digits and then a division, and the intermediate value is never written down anywhere the model can attend to it.

That is consistent with the account of Zhou et al. (2023), under which the tasks a transformer extrapolates are those expressible as short RASP programs, and with the direct finding of Lauter et al. (2024) that circular regression and sequence-to-sequence transformers both largely fail at modular multiplication. The measurement here adds the controlled comparison those results did not have. The architecture, the data, the optimiser and the seed are held fixed, and the operation is the only thing that changes. The failure is therefore attributable to the operation and not to any choice of representation.

7.6 The hypotheses

H1 (place value generalises) holds, and the effect is small. On both rungs where anything is learned, the abacus arms answer some out-of-distribution problems and the positional arms answer none. On plain addition that is 1.4% and 2.1% against 0.0%, with intervals that do not meet. Describing a digit by what it is worth is worth something beyond the training widths. It is worth one digit, not a scale.

H2 (re-injection helps) fails, and on the harder rung it fails badly. Per-layer injection is statistically indistinguishable from a single input injection on plain addition and far worse on modular addition, 24.9% against 76.0% in distribution with disjoint intervals. Section 7.2 gives the reading. This is the architecture the present work submitted to the competition, and the measurement says it was the wrong choice.

H3 (the operation binds, not the encoding) holds. Modular multiplication reduces every arm to the same place: 1.8% to 3.2% in distribution with overlapping intervals, and 0 of 4,000 beyond. No encoding tested here brings the competition's own operation within reach.

H4 (in-distribution accuracy does not buy extrapolation) holds. Three arms sit at exactly 100.0% across the training widths and then at 0.0%, 1.4% and 2.1% beyond them. The in-distribution figure does not order the arms the way the out-of-distribution figure does, which is why both appear in every table.

H5 (the table must be trained wider than the data) holds on the rung where anything generalises, and is beside the point elsewhere. Training the significance embedding beyond the operand widths raises out-of-distribution accuracy on plain addition from 2.1% to 38.3%, with disjoint intervals, while leaving in-distribution accuracy at 100.0% (Section 7.4). On modular addition it trades in-distribution accuracy away for no gain, and on modular multiplication it changes nothing. The hypothesis was formed after the first four arms had run, which is why it is reported last and why its qualification is given the same weight as its confirmation.

8 Limitations

8.1 Scale

These models have roughly 0.4 million parameters and train for 6,000 steps on CPU. The competition's regime (Section 2.4) is three orders of magnitude larger in operand width and was approached by others with far more computation. Nothing here shows that a large model trained long cannot do what these cannot; it shows where the boundary sits for a controlled comparison at this scale, and that the boundary falls on the operation rather than on the encoding.

8.2 Grokking is not tested

Power et al. (2022) report transitions after 10^5 optimisation steps, more than an order of magnitude beyond the budget here. A grokking transition on the harder rungs is therefore not excluded by these measurements. What the ladder does establish is the ordering of difficulty under a fixed budget, which is a different claim and a weaker one.

8.3 Scratchpads are held fixed

The repository implements algorithmic scratchpads, and they are a live route to the sequential computation these models lack (Nye et al., 2021). They are not varied here because varying them alongside the encoding would confound the comparison this paper exists to make. The interaction between place-value encoding and emitted working is unmeasured and is the obvious next experiment.

8.4 One base, one architecture

Everything is base ten with a three-layer decoder. Base is a free parameter in the competition, declared per submission, and a larger base shortens every number and therefore every carry chain. Whether the encoding comparison holds across bases is not tested here.

8.5 One modality

The question this paper asks is whether a representation carries to inputs longer than those it was trained on, and it is asked of digit strings only. The same question has a form in every modality where a structure has to be read off an input of unbounded extent, and the answer need not agree: a zero-shot video pipeline (Thakur and Talele, 2026) must rely on whatever its representation already carries, because there is no training signal at the point of use at all. Whether place value has an analogue that transfers in that setting is open, and this paper's result does not speak to it.

9 Conclusion

Describing a digit by what it is worth rather than by where it sits is the right idea, and it is worth far less than it should be unless the description itself is learned. Five encodings on one architecture, one dataset and one seed separate those two statements. Place value alone carries plain addition about one digit past the training widths, 2.1% against 0 of $4{,}000$ for sequence position. Training the same significance embedding over a wider range than the operand widths provide carries it to 38.3%, with in-distribution accuracy untouched (Section 6.2). The difference between those two numbers is which rows of one table received gradients.

Two results cut the other way, and they bound what the first is worth. Re-injecting the significance signal at every layer, the architecture this work submitted to the competition, is no better than injecting it once and is far worse on the harder

rung. And modular multiplication, the operation the competition actually sets, defeats all five encodings equally: every arm sits between 1.8% and 3.2% inside the training widths and at zero outside them. The fix that transforms addition does nothing for it.

The honest summary is that the encoding question and the arithmetic question are separate, and only the first is settled here. A better description of place buys generalisation on a task a transformer can already do. It does not make multiplication with reduction into a task a transformer can do. On the evidence here that second problem is where the difficulty in this competition lives, which agrees with what Lauter et al. (2024) reported and locates it more precisely: not in the representation, and not in the length, but in the operation.

Section 9 lists the scripts that regenerate every table above.

Data and code availability

All code and data live at `https://github.com/Amey-Thakur/SAIR-MODULAR-ARITHMETIC-CHALLENGE` (Thakur, 2026c). The experiment in this paper is `src/experiments/length_generalization.py`, which writes its results as JSON; the architectures are under `src/architectures/`, and the submitted model is published separately on the Hugging Face Hub. Every configuration is seeded, so the twelve runs behind every table are the same twelve on any machine. Reproducing the paper needs Python 3.10 or later and PyTorch 2.0 or later, and no GPU.

References

A. Alfarano, F. Charton, Y. Jia, K. Lauter, C. Li, T. Tao, and E. Wenger. SAIR Modular Arithmetic Challenge. SAIR Foundation, 8 June to 12 August 2026. `https://competition.sair.foundation/competitions/modular-arithmetic-challenge/overview`. Sets the problem, fixes exact match as the score, and defines the sandbox that forbids computing the answer by any means other than learned parameters, which is what makes the task a test of a model rather than of a library.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017. arXiv:1706.03762. The architecture every arm here shares, and the source of the absolute positional embedding that Section 2.2 argues is the wrong description for a digit.

A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: generalization beyond overfitting on small algorithmic datasets. arXiv:2201.02177, 2022. Establishes that algorithmic tasks generalise long after the training loss settles. This paper takes from it the reason its own budget is a stated limitation rather than a claim of impossibility.

N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. arXiv:2301.05217. Identifies the Fourier construction a transformer learns for modular addition at a fixed modulus. This paper takes from it the reason a variable modulus is the harder problem: the circuit it describes cannot be built when the modulus is an input.

S. McLeish, A. Bansal, A. Stein, N. Jain, J. Kirchenbauer, B. R. Bartoldson, B. Kailkhura, A. Bhatele, J. Geiping, A. Schwarzschild, and T. Goldstein. Transformers can do arithmetic with the right embeddings. arXiv:2405.17399, 2024. Introduces abacus embeddings, which describe a digit by its significance. This paper takes the idea directly, adds the per-layer injection variant, and asks what it is worth once a modulus is involved.

J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu. RoFormer: enhanced transformer with rotary position embedding. arXiv:2104.09864, 2021. The rotary encoding used as the second positional baseline, included because a relative scheme is the fairest positional comparison for a task whose inputs change length.

O. Press, N. A. Smith, and M. Lewis. Train short, test long: attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022. arXiv:2108.12409. Frames extrapolation to unseen lengths as a property of the positional scheme rather than of scale, which is the framing this paper tests in the modular setting.

A. Ruoss, G. Delétang, T. Genewein, J. Grau-Moya, R. Csordás, M. Bennani, S. Legg, and J. Veness. Randomized positional encodings boost length generalization of transformers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023. arXiv:2305.16843. Shows that breaking the binding between behaviour and particular indices improves extrapolation, which is the same diagnosis this paper makes and answers with significance rather than randomisation.

H. Zhou, A. Bradley, E. Littwin, N. Razin, O. Saremi, J. Susskind, S. Bengio, and P. Nakkiran. What algorithms can transformers learn? A study in length generalization. arXiv:2310.16028, 2023. Predicts which tasks extrapolate

from whether they admit a short RASP program. This paper takes from it the expectation that the multiplication rung should be hardest, which Section 7 tests.

F. Charton. Learning the greatest common divisor: explaining transformer predictions. In *International Conference on Learning Representations*, 2024. arXiv:2308.15594. Characterises completely what a transformer trained on a number-theoretic function actually learns. This paper takes from it the caution that apparent competence at a number-theoretic task may rest on a much simpler rule that agrees with it on the training distribution.

E. Wenger, M. Chen, F. Charton, and K. Lauter. SALSA: attacking lattice cryptography with transformers. In *Advances in Neural Information Processing Systems 35*, 2022. arXiv:2207.04785. Trains transformers on modular arithmetic as a component of a cryptanalytic attack, which is why the difficulty measured here is of interest beyond the competition that prompted it.

K. Lauter, C. Y. Li, K. Maughan, R. Newton, and M. Srivastava. Machine learning for modular multiplication. arXiv:2402.19254, 2024. Reports that both circular regression and sequence-to-sequence transformers largely fail at modular multiplication, and reads that as evidence for the hardness cryptography relies on. This is the closest prior result to the present paper, and Section 7 states precisely how the measurements here relate to it.

D. D. Africa, S. M. Kapoor, T. S. Sorg, and C. Mishra. Learning modular exponentiation with transformers. In *Advances in Neural Information Processing Systems 38*, 2025. arXiv:2506.23679. Finds interpretable circuits for modular exponentiation concentrated in the final attention layer, and sudden generalisation across related moduli. Evidence that modular tasks are learnable in principle, which sharpens the question of why the multiplication rung here is not.

M. Nye, A. J. Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, C. Sutton, and A. Odena. Show your work: scratchpads for intermediate computation with language models. arXiv:2112.00114, 2021. Establishes that emitting intermediate state buys multi-step arithmetic accuracy. Held fixed here to keep the encoding comparison clean, and named in Section 8 as the obvious next variable.

J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35*, 2022. arXiv:2201.11903. The prompting counterpart to an emitted scratchpad, cited for the same reason: computation spent in tokens substitutes for computation the architecture cannot perform in depth.

A. Thakur. Chebotarev fingerprints: identifying degree-24 Galois groups without computer algebra. 2026. https://github.com/Amey-Thakur/SAIR-INVERSE-GALOIS-PROBLEM-IGP24. Bounds what a statistical classifier can separate before any data is seen, and finds the limit in the geometry of the candidate set. This paper looks for the analogous limit and finds it in the operation rather than in the representation, which is the comparison Section 7 draws.

A. Thakur. Frame-synchronous hand gesture detection by projected winding order. arXiv:2609.13269, 2026. Identifies a class from a deterministic structural signature rather than a learned representation. The significance of a digit is such a signature: it is computed, not learned, and it is what the abacus arms are given.

A. Thakur, K. Dhiman, and M. Phansikar. Neuro-fuzzy: artificial neural networks and fuzzy logic. *International Journal for Research in Applied Science and Engineering Technology*, 9(9):128–135, 2021. 10.22214/ijraset.2021.37930. Treats inference under graded confidence. Relevant here by contrast: exact match admits no graded answer, which is what Section 2.1 argues makes this task unlike most sequence prediction.

A. Thakur and A. Konde. Fundamentals of neural networks. *International Journal for Research in Applied Science and Engineering Technology*, 9(VIII):407–426, 2021. 10.22214/ijraset.2021.37362. Supplies the background this paper works in: the decoder, the embedding table and the optimiser that every arm holds fixed.

A. Thakur. SAIR Modular Arithmetic Challenge: architectures, curriculum and sandbox. Software and dataset, 2026. https://github.com/Amey-Thakur/SAIR-MODULAR-ARITHMETIC-CHALLENGE. The artefact for this paper: the four architectures, the data generators, the scratchpad implementation, the sandbox validator, and the experiment that produces every table below.

A. Thakur and M. Satish. Clock synchronization in distributed systems. *International Research Journal of Engineering and Technology*, 9(3), 2022. https://www.irjet.net/archives/V9/i3/IRJET-V9I3350.pdf. Modular arithmetic is the arithmetic of a clock face, and this is the setting where the wrap-around that makes it modular has to be handled explicitly.

A. Thakur, M. Satish, R. K. Kahlon, H. Rizvi, and A. Davare. QuadTree visualizer. *International Journal of Engineering Research and Technology*, 11(4), 2022. 10.5281/zenodo.18447415. Addresses a point by recursive subdivision, so each digit of the address names a scale rather than an offset: the same construction the significance embedding applies to the digits of a number.

A. Thakur and M. Satish. Generative adversarial networks. *International Journal for Research in Applied Science and Engineering Technology*, 9(8):2307–2325, 2021. 10.22214/ijraset.2021.37723. The adversarial objective this paper does not use, and the contrast that makes exact match available: a likelihood-trained decoder is calibrated at each position, an adversarially trained generator need not be.

A. Thakur, H. Rizvi, and M. Satish. White-box cartoonization using an extended GAN framework. *International Journal of Engineering Applied Sciences and Technology*, 5(12), 2021. 10.33564/IJEAST.2021.v05i12.049. Decomposes an image into surface, structure and texture before translating it: what transfers is decided by the representation, not the capacity.

A. Thakur and M. Satish. Adversarial open domain adaption framework (AODA): sketch-to-photo synthesis. *International Journal of Engineering Applied Sciences and Technology*, 6(2), 2021. 10.33564/IJEAST.2021.v06i02.037. Adaptation to inputs outside the training domain, the image-side analogue of the length generalisation measured here.

A. Thakur and S. Talele. A modular zero-shot pipeline for accident detection, localization, and classification in traffic surveillance video. arXiv:2604.09685, 2026. 10.48550/arXiv.2604.09685. Operates with no task-specific training, so its answers rest entirely on what the representation already carries: the multimodal form of the question asked here.